Chương 5

Mạng nơ-ron tích chập

5.1 Giới thiệu mạng nơ-ron tích chập

5.1.1 Lịch sử mạng nơ-ron tích chập

Mạng Nơ-ron tích chập (Convolutional Neural Network, còn được viết tắt là CNN hay ConvNet) là một trong những nền tảng quan trọng của chuyên ngành *Thị giác máy tính* (Computer Vision) nói riêng, hay của ngành Học sâu nói chung. CNN được chuyên dùng cho các bài toán liên quan đến hình ảnh như phân loại, phân tích hình ảnh (image classification) hay nhận diện khuôn mặt (facial recognition). Tác giả của kiến trúc này chính là nhà nghiên cứu lỗi lạc Yann LeCun, người từng đạt giải Turing năm 2018. Ông đã trình bày ý tưởng của mình thông qua bài báo *Lan truyền ngược áp dụng cho nhận dạng chữ viết tay cho zipcode* (Backpropagation applied to handwritten zip code recognition) (LeCun et al., 1989).

Những ý tưởng sơ khai đầu tiên để hình thành nên một mạng nơ-ron tích chập như hiện nay đã bắt nguồn từ những năm 50 và 60 của thế kỷ trước. Trong một bài báo khoa học của hai nhà nghiên cứu David H. Hubel và Torsten Wiesel về những vùng cảm thụ của vỏ não thị giác (receptive

fields of visual cortex), tác giả đã nêu ra cách xử lý hình ảnh của các tế bào thị giác chính là nhận biết các *cạnh thẳng* (straight edges). Các tế bào thị giác được chia làm hai loại, đó là *tế bào tối giản* (simple cell) và *tế bào phức hợp* (complex cell). Hai nhà khoa học cũng đưa ra mô hình xếp tầng (cascading model) để lý giải cách nhận dạng mẫu (pattern recognition) của hai loại tế bào này (Hubel and Wiesel, 1959).

Đến năm 1979, nhà khoa học dữ liệu người Nhật Bản Kunihiko Fukushima đưa ra khái niệm "Neocognitron", đây được xem là mạng Nơ-ron tích chập nguyên thủy của ngành Thị giác máy tính. Dựa trên những ý tưởng của Hubel và Wiesel, các *tầng* (layer) của Neocognitron được chia làm hai loại cơ bản: tầng S-cells được sử dụng để trích xuất các *thuộc tính cục bộ* (local feature) của hình ảnh, tương đương với simple cell (dùng để xử lý các đường thẳng cơ bản); trong khi C-cells được dùng để giảm thiểu các sai sót khi kết hợp nhiều thuộc tính cục bộ với nhau (tương đương với complex cell). Các *thuộc tính cục bộ* được tích hợp dần và phân loại ở các tầng sâu hơn. Từ đó máy tính sẽ nhận dạng được các đặc điểm khác nhau của hình ảnh trong quá trình học và có thể phân loại được hình ảnh dựa trên các *bộ lọc* khác nhau (filter). Đến nay thì những ý tưởng cơ bản của các mạng CNN hiện đại vẫn dựa trên mạng Neocognitron này (Fukushima, 1980).

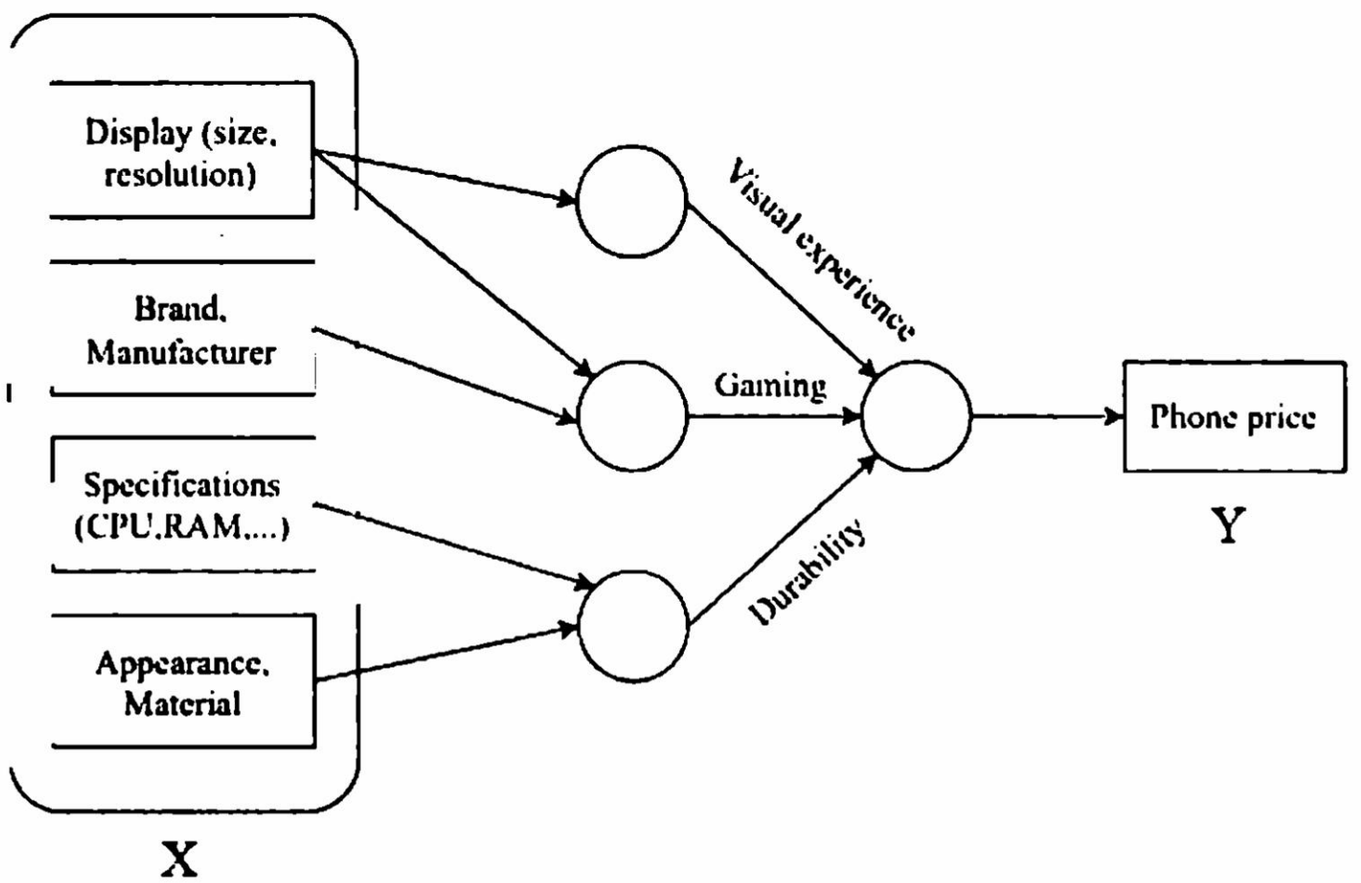
Có thể thấy, những ý tưởng khởi phát của CNN đã có từ cách đây khá lâu (gần 30 năm), nhưng sự trỗi dậy của ngành Học sâu, cụ thể hơn là Thị giác máy tính chỉ thực sự bắt đầu từ những năm đầu của thế kỷ XXI với sự phát triển của ngành công nghiệp Trò chơi điện tử. Các card đồ họa ngày càng được nâng cấp để nâng cao trải nghiệm chơi game cho người dùng, mà cốt lõi chính là việc tăng cường khả năng tính toán của các bộ xử lý đồ họa GPU.

Như đã trình bày, có vẻ như CNN và GPU thuộc hai chuyên ngành khác biệt, nhưng thực tế chúng đều hoạt động dựa trên các thao tác với ma trận (matrix manipulation). Hình ảnh là một ma trận các điểm

ảnh (pixel), còn video thì ghép nối nhiều hình ảnh khác nhau trong một khoảng thời gian nhất định (60 frame per second chẳng hạn) để tạo nên những chuyển động. Do tính chất này, GPU được cải thiện sao cho việc tính toán các ma trận pixel được diễn ra nhanh nhất có thể, điều này vô tình khiến các mạng nơ-ron nhân tạo được hưởng lợi rất nhiều.

5.1.2 Đặc tính cơ bản của mạng nơ-ron tích chập

Vậy tại sao mạng nơ-ron tích chập lại được sử dụng phổ biến cho các bài toán phân tích hình ảnh? Câu trả lời đến từ một bản chất tự nhiên của CNN, đó là khả năng *rút trích thuộc tính ẩn* (latent feature extraction) cực kỳ tốt. Để có thể nhận biết rõ hơn về tầm quan trọng của việc rút trích được những thuộc tính ẩn, ta hãy cùng phân tích ví dụ dưới đây về mối liên hệ giữa cách thức con người suy nghĩ để đưa ra quyết định và quá trình học của mạng nơ-ron nhân tạo.



Hình 5.1: Phương thức phân tích dữ liệu và dự đoán giá sản phẩm điện thoại thông minh

Hình 5.1 là một minh họa về quá trình phân tích và rút trích các thuộc tính ẩn để từ đó đưa ra quyết định định giá sản phẩm của một nhà phân phối điện thoại thông minh. Có thể thấy được giá trị đầu vào chính là những thông số thường được đi kèm với mỗi dòng điện thoại như màn hình, vi xử lý, RAM, hãng sản xuất,... Tuy nhiên, các thông số này không trực tiếp quyết định số tiền mà người dùng cần bỏ ra để sở hữu chiếc điện thoại. Bản thân người dùng khi mua một sản phẩm luôn có những yêu cầu nhất định và phù hợp cho cá nhân mình, chẳng hạn như độ bền, khả năng chiến game, trải nghiệm hiển thị,...

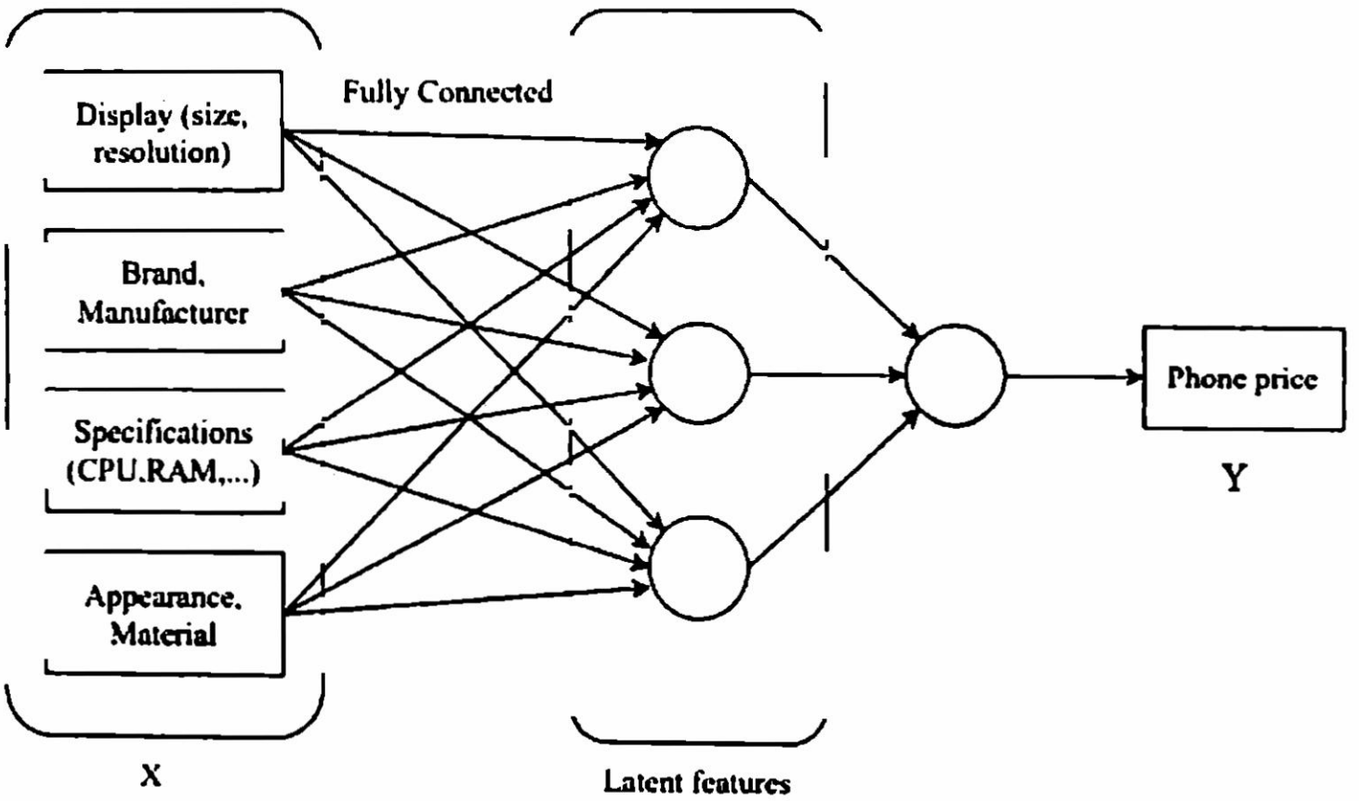
Như vậy, đối với nhà phân phối, họ cần rút trích được các thuộc tính tiềm ẩn của sản phẩm và có ý nghĩa đối với khách hàng của họ (cũng chính là nhu cầu người dùng) từ những thông số đầu vào tương đối vô nghĩa. Ví dụ như thông số màn hình (kích thước, độ phân giải, tấm nền) sẽ quyết định trải nghiệm thị giác; thông số màn hình cộng với thông số phần cứng như chip xử lý, bộ nhớ RAM sẽ quyết định trải nghiệm chơi game; hoặc hãng sản xuất, chất liệu và thiết kế của sản phẩm có thể ảnh hưởng đến độ bền. Từ đó, nhà phân phối đưa ra giá trị cho mỗi loại điện thoại sao cho phù hợp và bán chạy khi giới thiệu đến người dùng.

Hình 5.2 mô tả cơ bản quá trình học của một *mạng nơ-ron nhân tạo* (neural network). Từ các giá trị đầu vào, mạng nơ-ron tiến hành các thao tác xử lý để trích xuất các *thuộc tính tiềm ẩn* (latent features) của dữ liệu. Các thuộc tính này có thể tiếp tục được làm đầu vào cho các lớp sau đó cho đến khi tìm được kết quả cuối cùng.

Một điểm đặc biệt của các thuộc tính tiềm ẩn được trích xuất từ mạng nơ-ron nhân tạo là thậm chí con người đôi khi cũng không thể, hoặc rất khó khăn để suy luận ra được. Một ví dụ đơn giản chính là cách quá trình trẻ nhỏ nhận biết các con vật khác nhau. Dù thực tế chúng ta không được học về một định nghĩa hay lý thuyết cụ thể để phân biệt các loài vật, nhưng não bộ ta đã học được điều đó trong quá trình tiếp xúc trực tiếp hay gián tiếp. Do đó, khi được hỏi hãy liệt kê cụ thể các đặc

điểm để phân biệt chó và mèo, đôi khi chúng ta cũng gặp không ít khó khăn để trích xuất các đặc điểm này, việc mà các mạng nơ-ron nhân tạo được thực hiện bởi máy tính lại thực hiện rất tốt.

Có nhiều mạng nơ-ron có thể đảm nhiệm tốt khả năng trích xuất thuộc tính. Tuy nhiên, đối với một số mạng nơ-ron sử dụng cơ chế *kết nối đầy đủ* (fully-connected), có một vấn đề nảy sinh là số *trọng số* (parameter) cần học là rất lớn bởi vì tất cả các giá trị đầu vào đều tham gia vào quá trình học của mỗi trọng số. Điều này có thể khiến cho quá trình huấn luyện diễn ra tương đối chậm. Với mạng nơ-ron tích chập, cơ chế *chia sẻ trọng số* (shared weight) sẽ giúp tiết kiệm đáng kể chi phí với trọng số được học giảm rất nhiều. Chúng ta sẽ cùng phân tích về cơ chế chia sẻ trọng số này kỹ hơn ở phần sau của chương.



Hình 5.2: Quá trình học và trích xuất thuộc tính của một mạng nơ-ron dự đoán giá sản phẩm điện thoại thông minh.

5.2 Cách thức hoạt động của mạng nơ-ron tích chập

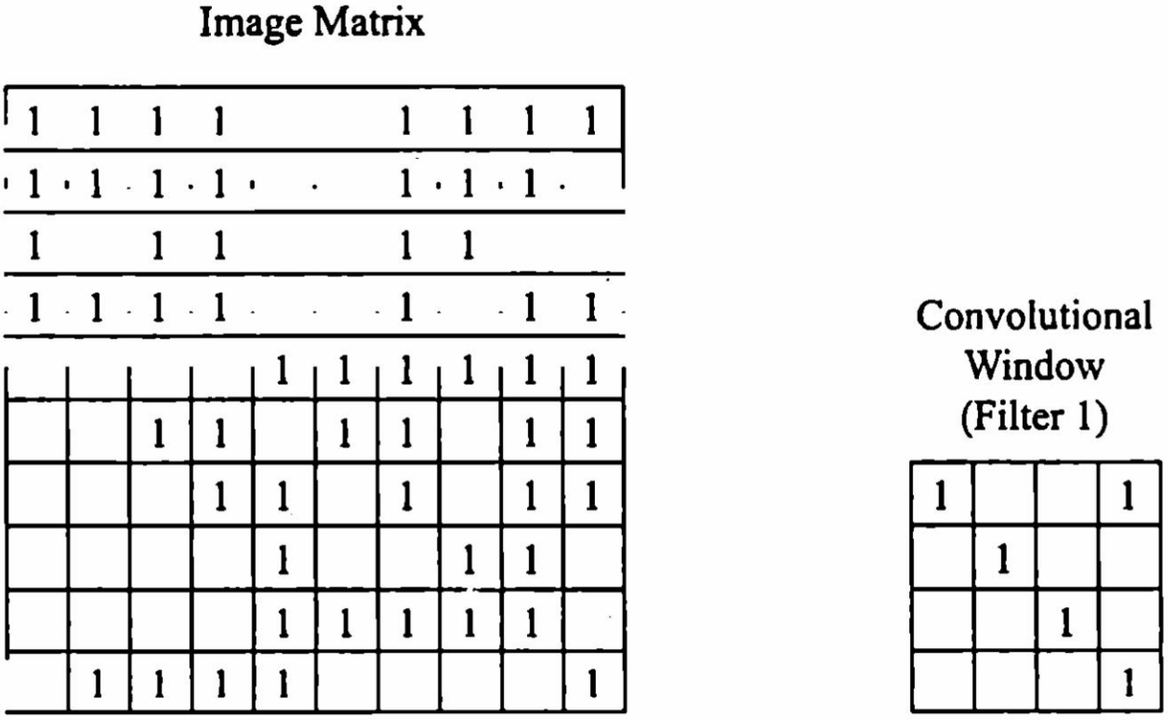
5.2.1 Phép tích chập (Convolution)

Dúng như cái tên của CNN, phép tính *tích chập* (convolution) chính là đặc trưng mạng nơ-ron này. Để có thể dễ dàng nắm bắt được ý tưởng của phép tích chập, ta hãy cùng phân tích ví dụ sau đây.

Xét một hình ảnh được biểu diễn dưới dạng ma trận I kích thước 10x10 gồm các chữ số 1 (các vị trí không có số 1 được hiểu là 0) và một cửa sổ tích chập K (cửa sổ trượt) là ma trận 4×4 như Hình 5.3. Ta có công thức tính một phần tử của ma trận kết quả từ phép tích chập như sau:

$$S_{ij} = (I * K)_{ij} = \sum_m \sum_n I(m,n)K(i-m, j-n) \tag{5.1}$$

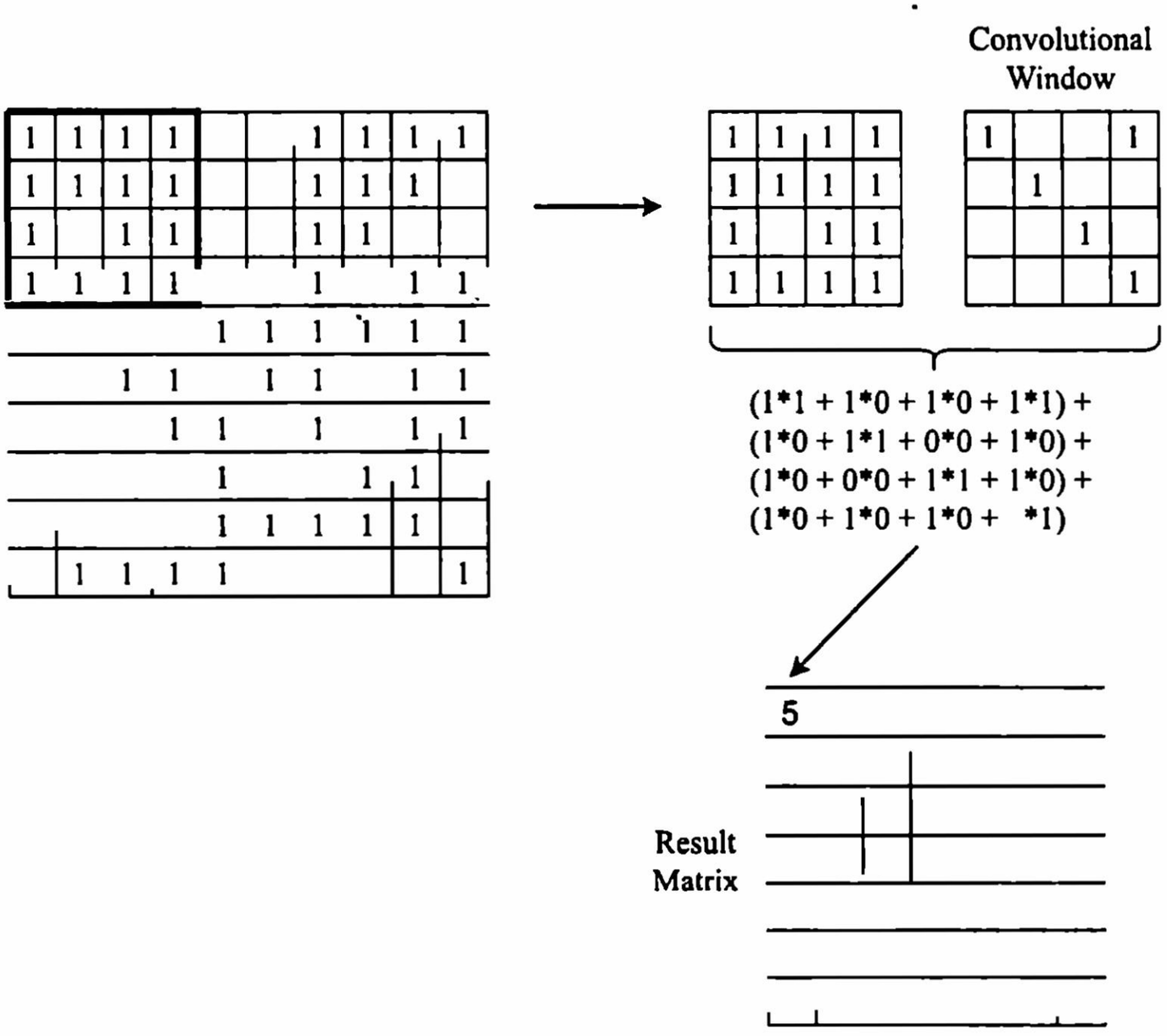
với i, j là vị trí dòng và cột của phần tử kết quả, m và n là kích thước của ma trận cửa sổ tích chập.



Hình 5.3: Ma trận hình ảnh (trái) và ma trận cửa sổ tích chập (phải)

Để dễ hình dung hơn về phép tích chập, ta sẽ xét ví dụ trên bằng cách trực quan hóa thông qua hình ảnh. Ta thực hiện phép tích chập bằng cách trượt cửa sổ như sau:

- Nhân tích chập cửa sổ trượt với ma trận con 4x4 tại vị trí đầu tiên của ma trận hình ảnh. Phép nhân tích chập được thực hiện bằng cách nhân từng phần tử của ma trận hình ảnh con với phần tử tại ví trí tương ứng của cửa sổ trượt sau đó cộng các kết quả lại. Kết quả cuối cùng chính là giá trị của phần tử đầu tiên trong ma trận kết quả. Hình 5.4 mô tả cụ thể quá trình này.



Hình 5.4: Quá trình nhân tích chập cho phần tử đầu tiên

- Trượt cửa sổ tích chập sang phải của ma trận hình ảnh một đơn vị cột và tiếp tục nhân tích chập để tìm được phần tử thứ 2 trong ma trận kết quả. Tiếp tục trượt sang phải cho đến khi không thể trượt thêm. Như vậy ta có được các phần tử của hàng đầu tiên cho ma trận kết quả tích chập.

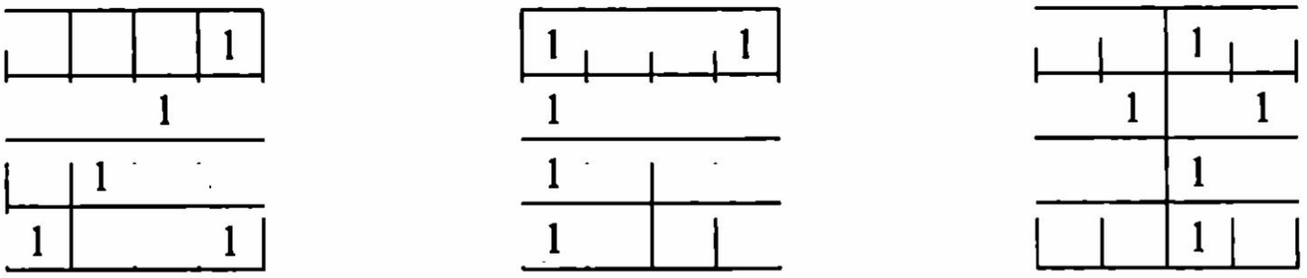
- Để tính tiếp hàng thứ 2 cho ma trận kết quả, ta dời cửa sổ trượt về vị trí ban đầu và trượt cửa sổ xuống một đơn vị hàng. Tiếp tục lặp lại quá trình nhân tích chập và trượt cửa sổ. Ma trận kết quả sau khi hoàn thành quá trình tích chập sẽ có kích thước 7x7 như Hình 5.5.

5	2	2	3	2	4	4
5	2	2	3	2	4	4
5	2	2	3	2	4	4
5	2	2	3	2	4	4
5	2	2	3	2	4	4
5	2	2	3	2	4	4
5	2	2	3	2	4	4

Hình 5.5: Ma trận kết quả đầy đủ sau khi nhân tích chập với cửa sổ trượt đầu tiên

Như vậy, ta đã có thể nắm được quá trình nhân tích chập diễn ra như thế nào.

Để thực hành làm quen với phép tích chập, hãy tiếp tục nhân tích chập ma trận hình ảnh ban đầu cho các cửa sổ trượt 2,3 và 4 như trong Hình 5.6. Kết quả nhận được sẽ giống với 3 ma trận kết quả trong Hình 5.7.



Filter 2 Filter 3 Filter 4

Hình 5.6: 3 ma trận cửa sổ trượt tiếp theo

3	3	2	3	2	4	5
3	3	2	2	4	5	2
2	3	2	2	4	4	3
2	2	1	5	3	3	4
1	2	3	3	3	3	2
0	3	3	3	3	3	4
2	3	2	3	2	3	4

Kết quả tích chập sử dụng filter 2

5	3	3	5	1	1	5
4	3	0	4	2	2	4
3	2	3	3	3	2	3
2	2	2	3	3	3	4
0	1	1	2	5	3	3
0	2	2	1	4	3	3
1	2	1	3	4	2	3

Kết quả tích chập sử dụng filter 3

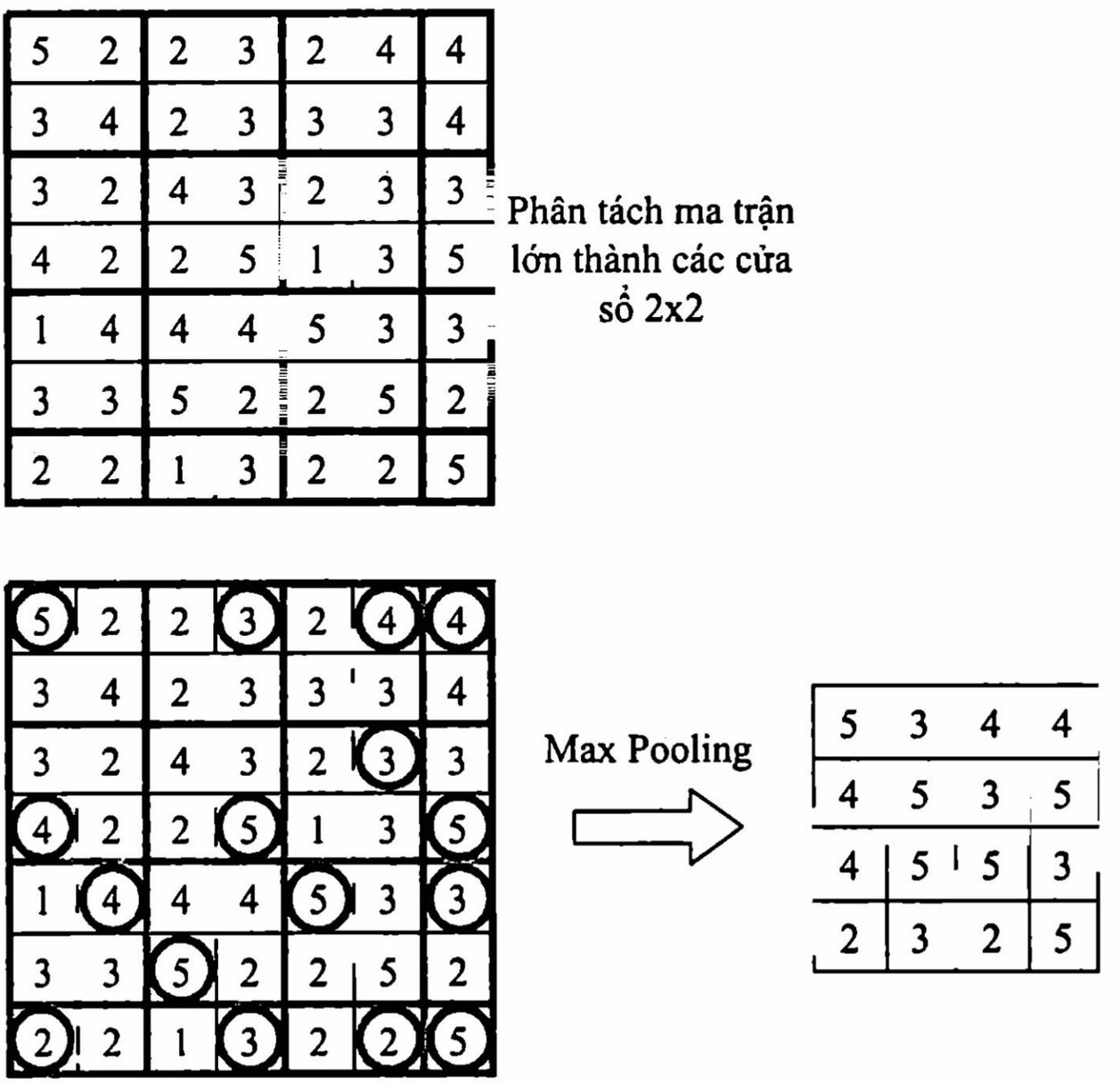
5	3	1	1	4	4	3
2	3	1	2	4	3	4
4	2	3	3	2	4	3
2	3	3	3	4	3	4
1	3	4	2	3	3	4
2	1	4	4	4	4	4
2	3	3	1	3	2	3

Kết quả tích chập sử dụng filter 4

Hình 5.7: 3 ma trận kết quả sau khi thực hiện phép nhân tích chập từ 3 cửa sổ trượt trên

5.2.2 Phép gộp (Pooling)

Sau khi có được các ma trận kết quả với kích thước 7x7 từ phép tính tích chập. Ta tiếp tục thực hiện *phép gộp* (pooling) để tiếp tục thu nhỏ ma trận hình ảnh. Có nhiều cách thực hiện phép gộp khác nhau như: lấy giá trị lớn nhất (max pooling), lấy giá trị trung bình (averrage pooling) hay lấy giá trị tổng (sum pooling). Trong thực tế max pooling có khả năng khử nhiễu tốt hơn các loại pooling khác, vì vậy ta sẽ sử dụng max pooling để làm ví dụ minh họa cho phép gộp.



Hình 5.8: Quá trình thực hiện phép gộp max pooling

Xét ma trận kết quả được tính từ cửa sổ tích gộp đầu tiên. Ta thực hiện phép gộp max pooling như sau (Hình 5.8 mô tả quá trình thực hiện phép gộp này):

- Chia ma trận kết quả ra thành các ma trận con (grid) có kích thước 2×2. Lưu ý là do ma trận kết quả có kích thước là 7×7 nên các ma trận con nằm ở cuối hàng và cuối cột chỉ có kích thước 1×2 hoặc 2×1.
- Giá trị lớn nhất trong mỗi ma trận con (khoanh tròn màu đỏ) chính là phần tử của ma trận gộp mới. Lúc này ma trận kết quả sau khi gộp chỉ còn có kích thước 4×4, giảm đáng kể so với ma trận 10×10 ban đầu.

Tiếp tục thực hiện phép gộp dùng giá trị lớn nhất cho 3 ma trận kết quả tích chập còn lại, ta nhận được 4 ma trận kết quả tương ứng với 4 cửa sổ trượt ban đầu như Hình 5.9.

5	3	4	4
4	5	3	5
4	5	5	3
2	3	2	5

Kết quả max pooling sử dụng filter 1

3	3	4	5
3	5	4	4
3	3	3	4
3	3	3	4

Kết quả max pooling sử dụng filter 2

5	5	2	5
3	3	3	4
2	2	5	3
2	3	4	3

Kết quả max pooling sử dụng filter 3

5	2	4	4
4	3	4	4
3	4	4	4
3	3	3	3

Kết quả max pooling sử dụng filter 4

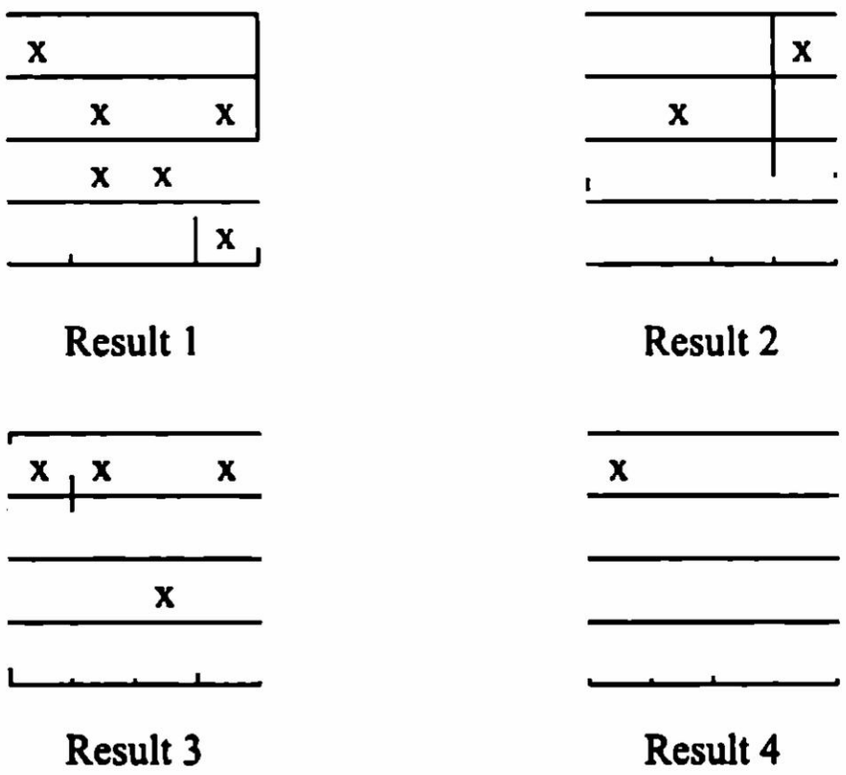
Hình 5.9: 4 ma trận kết quả sau khi thực hiện max pooling.

Như vậy, ta đã nắm bắt được ý tưởng của phép toán thứ 2 trong mạng nơ-ron tích chập đó là phép gộp (*pooling*).

5.2.3 Ý nghĩa của ma trận kết quả

Sau khi có được 4 ma trận kết quả thì dễ dàng nhận thấy rằng giá trị tối đa có được của mỗi phần tử là 5. Giá trị này chỉ có thể có khi toàn bộ các phần tử có giá trị là 1 trên cửa sổ filter cũng có giá trị là 1 tại vị trí tương ứng trên ma trận gốc, hay nói cách khác, toàn bộ hình ảnh trên cửa sổ filter đều xuất hiện toàn vẹn tại vị trí tương ứng trên cửa sổ gốc. Vì vậy, có thể nói rằng, số 5 thể hiện sự xuất hiện hoàn chỉnh của đối tượng được biểu diễn filter trên cửa sổ gốc.

Để thể hiện điều này, chúng ta sẽ tạo ra một cửa sổ hiển thị bằng cách chỉ biểu diễn các giá trị 5 (dưới dạng dấu × trên cửa sổ kết quả) như Hình 5.10.



Hình 5.10: 4 ma trận kết quả mới sau khi lượt giảm các giá trị nhỏ hơn 5

Có thể thấy, sau khi biến đổi thì các ma trận kết quả đang thể hiện được sự phân bố mẫu (pattern) của các cửa sổ trượt nhỏ trên ma trận hình ảnh ban đầu. Ví dụ như cửa sổ đầu tiên thể hiện một đường chéo các số 1 (xem lại Hình 5.4) thì ma trận kết quả thể hiện phân bố đúng như bản chất ma trận hình ảnh gốc, với một đường chéo chạy dài từ đầu đến

cuối. Hay như ma trận thứ 4, cửa sổ trượt thể hiện một hình tương tự mũi tên bị khuyết ở giữa và nằm lệch về bên phải (xem Hình 5.6). Ở ma trận gốc thì hình này chỉ xuất hiện đúng một lần tại góc trên cùng bên trái của hình ảnh.

Từ quan sát này, ta có thể hiểu được vì sao các cửa sổ trượt này được gọi là các *bộ lọc* (filter). Nhiệm vụ của chúng chính là tìm ra các đặc trưng của hình ảnh. Ở các lớp cao hơn, các bộ lọc đóng các vai trò khác nhau tùy theo yêu cầu của bài tóan như *bộ phát hiện cạnh* (edge detector), *bộ phát hiện nhiễu* (noise detector),...

5.2.4 Bản đồ thuộc tính

5	3	4	4
4	5	3	5
4	5	5	3
2	3	2	5
3	3	4	5
3	5	4	4
3	3	3	4
3	3	3	4
5	5	2	5
3	3	3	4
2	2	5	3
2	3	4	3
5	2	4	4
4	3	4	4
3	4	4	4
3	3	3	3

Hình 5.11: Bản đồ thuộc tính được tạo thành từ việc ghép các ma trận kết quả

Nếu ghép 4 ma trận kết quả ở Hình 5.9 lại với nhau, ta được một hình ảnh như Hình 5.11. Đây được gọi là *bản đồ thuộc tính* (feature map), được trích xuất sau quá trình tích chập của mạng CNN. Bản đồ thuộc tính là một đặc trưng nổi bật của mạng nơ-ron tích chập, bởi nó thể hiện được các *thuộc tính tiềm ẩn* (latent features) của hình ảnh đầu vào.

Cần lưu ý rằng *bản đồ thuộc tính* được sinh ra dựa trên các *bộ lọc* đã được định nghĩa trước (hay chính là các cửa sổ tích chập trong ví dụ trên). Do đó, điều quan trọng chính là làm sao định nghĩa được các *bộ lọc* này một cách phù hợp để áp dụng hiệu quả cho những bài toán cụ thể. Ví dụ như với bài toán nhận diện khuôn mặt người, *bộ lọc* sẽ là các bộ nhận diện mắt, mũi, miệng. Còn đối với bài toán nhận diện vạch kẻ đường, mạng nơ-ron cần học được các *bộ lọc* nhận diện đường thẳng,... Vấn đề này sẽ được thảo luận trong phần tiếp theo, khi ta cần sử dụng cơ chế học của *mạng nơ-ron nhân tạo* thông thường để có thể học được *bộ lọc*.

Như vậy, đối với lại con người thì bản đồ thuộc tính chỉ là một ma trận mới được tạo ra từ ma trận ban đầu sau khi thực hiện các phép tích chập và phép gộp. Tuy nhiên, đối với máy tính thì ma trận đặc trưng là bộ biểu diễn các thuộc tính tiềm ẩn được rút trích từ các ma trận trước.

Vì bản chất của ma trận đặc trưng là ma trận mới được tạo ra từ ma trận gốc nên có thể tiếp tục lặp lại quá trình tích chập và gộp thêm một số lần khác nữa để rút trích ra được các thuộc tính có tính trừu tượng cao hơn.

5.2.5 Cách áp dụng bộ lọc

Chúng ta xem xét một hình ảnh thực như Hình 5.12a có thể được biểu diễn dưới dạng mảng 2 chiều chứa các giá trị pixel. Dựa vào nội dung trình bày về phép tích chập trong Mục 5.2.1, ta có thể dễ dàng xây dựng bộ lọc phát hiện các đường thẳng hoặc đường ngang từ ảnh gốc.

Như Hình 5.12b, bộ lọc có ma trận cửa sổ trượt được chọn có kích thước 3×3 dùng để lọc các đối tượng có dạng đường thẳng có trong hình gốc. Và ngược lại, Hình 5.12c là bộ lọc với ma trận cửa sổ trượt 3×3 được dùng để lọc ra và hiển thị tất cả các đối tượng có dạng đường ngang xuất hiện trong hình gốc.



(a) (b) (c)

$$\begin{bmatrix} -1 & 0 & -1 \\ -1 & 0 & -1 \\ -1 & 0 & -1 \end{bmatrix} \qquad \begin{bmatrix} -1 & -1 & -1 \\ 0 & 0 & 0 \\ 1 & 1 & 1 \end{bmatrix}$$

Hình 5.12: Trích xuất đặc trưng của hình ảnh theo chiều dọc (b) và chiều ngang (c)

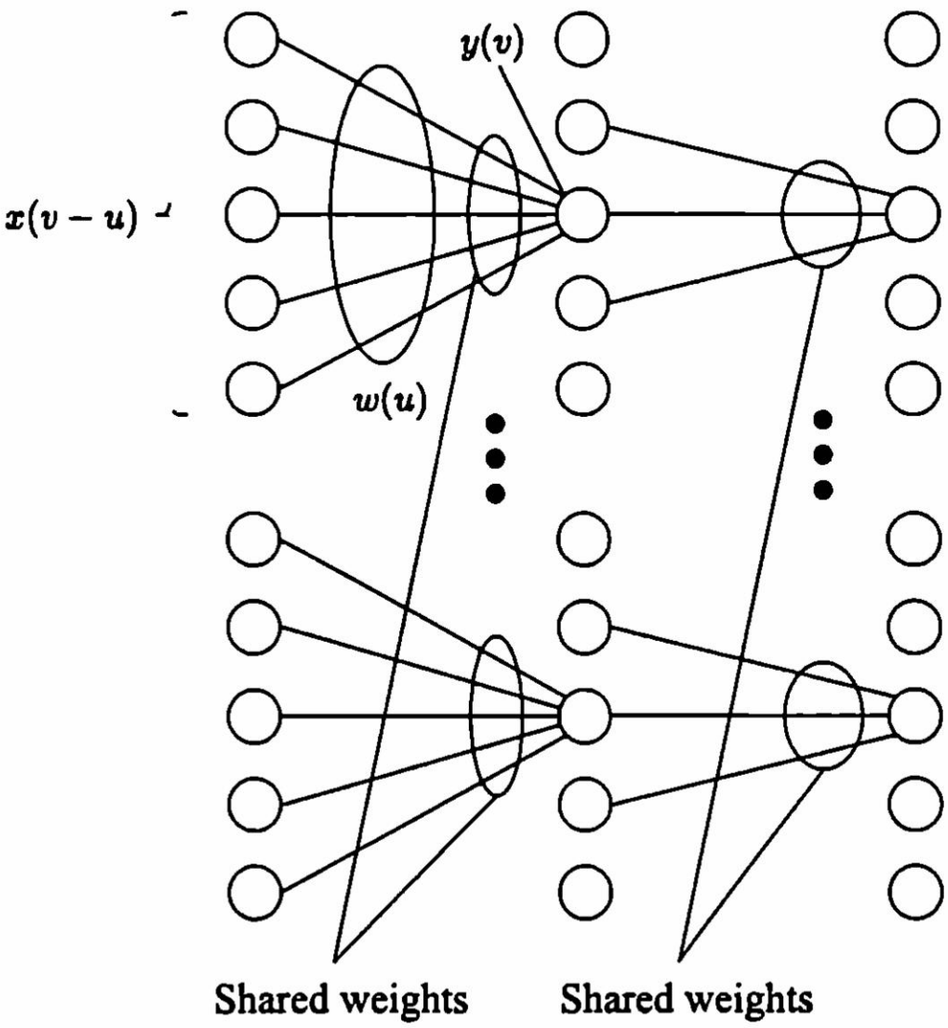
Như vậy, tùy vào mục đích của bộ lọc mà ta cần thiết kế và chọn lựa ma trận cửa sổ trượt thích hợp nhằm trích xuất được đặc trưng ứng vưới mục tiêu đề ra.

5.3 Mạng nơ-ron tích chập dưới góc nhìn của một mạng nơ-ron nhân tạo

Trong một mạng nơ-ron nhân tạo thông thường thì được cấu thành bởi các nút nơ-ron khác nhau nối tiếp nhau và qua quá trình xử lý thông tin sẽ tạo ra các nút nơ-ron ở tầng tiếp theo. Tương tự như vậy với mạng nơ-ron tích chập, ví dụ có một hình ảnh được biểu diễn dưới dạng ma trận đầu vào với kích thước 10 × 10 và một cửa sổ tích chập là ma trận 4 × 4 thì quá trình nhân tích chập tại vị trí đầu trên ma trận đầu vào

10×10 sẽ tạo ra được một phần tử của ma trận mới và đó cũng tương đương với một nút nơ-ron được tạo ra sau khi thực hiện một phép biến đổi. Nút nơ-ron mới sẽ nối với 16 điểm trên ma trận đầu vào.

Hình 5.13 mô tả một ma trận đầu vào cùng với một cửa sổ và khi thực hiện phép tích chập trên một vị trí trên ma trận đầu vào sẽ tạo ra được một nút nơ-ron. Khi cho cửa sổ trượt qua hàng thứ hai sẽ thu được một nơ-ron khác và khi ta cho ma trận lọc này trượt hết trên ma trận đầu vào thì sẽ tạo ra được 49 nút nơ ron khác nhau và mỗi nút nối với một ma trận 16 đơn vị.



Hình 5.13: Quá trình nhân tích chập một bộ lọc tạo ra các nơ-ron

Một đặc điểm quan trọng của mạng nơ-ron tích chập là *cơ chế chia sẻ trọng số* (shared weights). Có nghĩa là các trọng số trên mỗi bộ lọc phải giống nhau và các nơ-ron trong lớp ẩn đầu sẽ phát hiện chính xác điểm tương tự chỉ ở các vị trí khác nhau trong dữ liệu đầu vào. Việc làm này

sẽ làm giảm tối đa số lượng các *tham số* (parameters), mỗi bản đồ đặc trưng sẽ giúp phát hiện thêm một vài đặc trưng khác. Với một ma trận hình ảnh đầu vào kích thước 10×10 như ở trên và 4 bộ lọc có ma trận kích thước 4×4 thì mỗi bản đồ thuộc tính cần $4 \times 4 = 16$ trọng số và số nơ-ron được tạo ra ở lớp thứ hai là 49. Như vậy nếu có 4 bản đồ thuộc tính thì có $4 \times 16 = 64$ tham số. Với một mạng nơ-ron có kết nối đầy đủ thì chúng ta sẽ có $10 \times 10 \times 49 = 4900$ trọng số. Từ kết quả cho thấy sử dụng lớp tích chập sẽ cần số lượng tham số ít hơn nhiều lần so với lớp kết nối đầy đủ nhưng vẫn có thể rút ra các đặc trưng một cách hiệu quả.

Một khả năng khác của mạng nơ-ron tích chập là số tham số không phụ thuộc vào kích thước của đầu vào. Với những ma trận đầu vào có kích thước khác nhau và thông qua quá trình học theo phương pháp nơ-ron tích chập sẽ rút ra những thuộc tính ẩn mà ta có thể khó nhận thấy. Xét một ví dụ như sau: chúng ta có 10 bộ lọc và mỗi một bộ lọc sẽ là một ma trận kích thước $3 \times 3 \times 3$ và có một giá trị sai lệch (bias) là 1, chúng ta sẽ cần huấn luyện bao nhiêu parameter từ mạng nơ-ron tích chập này?

Để giải được bài toàn này, chúng ta sẽ tính số lượng parameters cần dùng mỗi bộ lọc rồi từ đó tính được kết quả như sau:

- Số lượng tham số cho mỗi bộ lọc là 3x3x3 = 27
- Tổng số tham số cho mỗi bộ lọc là 27 +1 = 28
- Tổng số tham số cho 10 bộ lọc là 28 x 10 = 280

Qua ví dụ trên, có thể rút ra nhận xét rằng số lượng tham số được tạo ra bởi mạng CNN này luôn luôn là 280, không phụ thuộc vào độ lớn của dữ liệu đầu vào.

5.4 Triển khai một mạng CNN

Khi triển khai trong một kiến trúc mạng end-to-end hoàn chỉnh thì một mạng CNN cơ bản bao gồm 3 bộ phận chính: *Tầng tích chập* (Convolu-

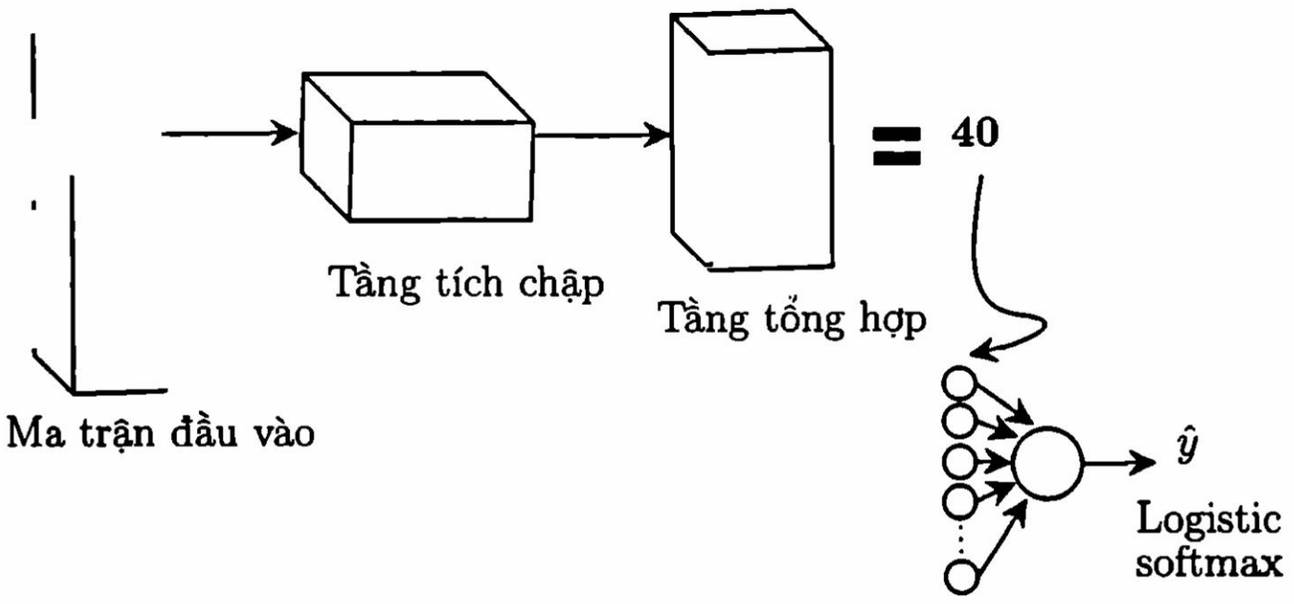
tion), *tầng tổng hợp* (Pooling), *tầng kết nối đầy đủ* (Fully Connected) có cách thức hoạt động giống như tầng kết nối trong mạng nơ-ron đã mô tả trong Chương 3. Trong đó:

- Tầng tích chập là tầng quan trọng nhất và cũng là tầng đầu tiên của của mô hình CNN, có chức năng chính là phát hiện các đặc trưng cụ thể của input ban đầu. Tầng này có các bộ phận chính là một ma trận đầu vào, bản đồ thuộc tính và các cửa sổ trượt là các khái niệm đã được trình bày trong Mục 5.2.
- Tầng tổng hợp có mục đích làm giảm số lượng thông số mà ta phải tính toán, điều này giúp giảm thời gian tính toán. Có 2 phương pháp tổng hợp thường gặp nhất là phép tổng hợp lớn nhất *(Max pooling)* và phép tổng hợp trung bình *(Average Pooling)*.

Vậy tại sao tầng cuối phải là kết nối đầy đủ? Trong thực tế, mạng CNN phải được huấn luyện như một hàm mục tiêu cụ thể để giải quyết một bài toán cụ thể nên tầng cuối đóng vai trò của việc đưa ra kết quả học và tính được giá trị của hàm mất mát, từ đó mạng CNN có thể học và cập nhật lại bộ trọng số của mạng bằng phương pháp lan truyền ngược.

Như vậy, vai trò của một tầng kết nối đầy đủ là giúp cho mạng CNN có thể học được các trọng số trên các cửa sổ trượt. Nếu chỉ có tầng tích chập và tầng tổng hợp thì mạng CNN không thể biết được cần phải học những gì cũng như không có cơ sở nào để có thể thực hiện việc cập nhật lại bộ trọng số.

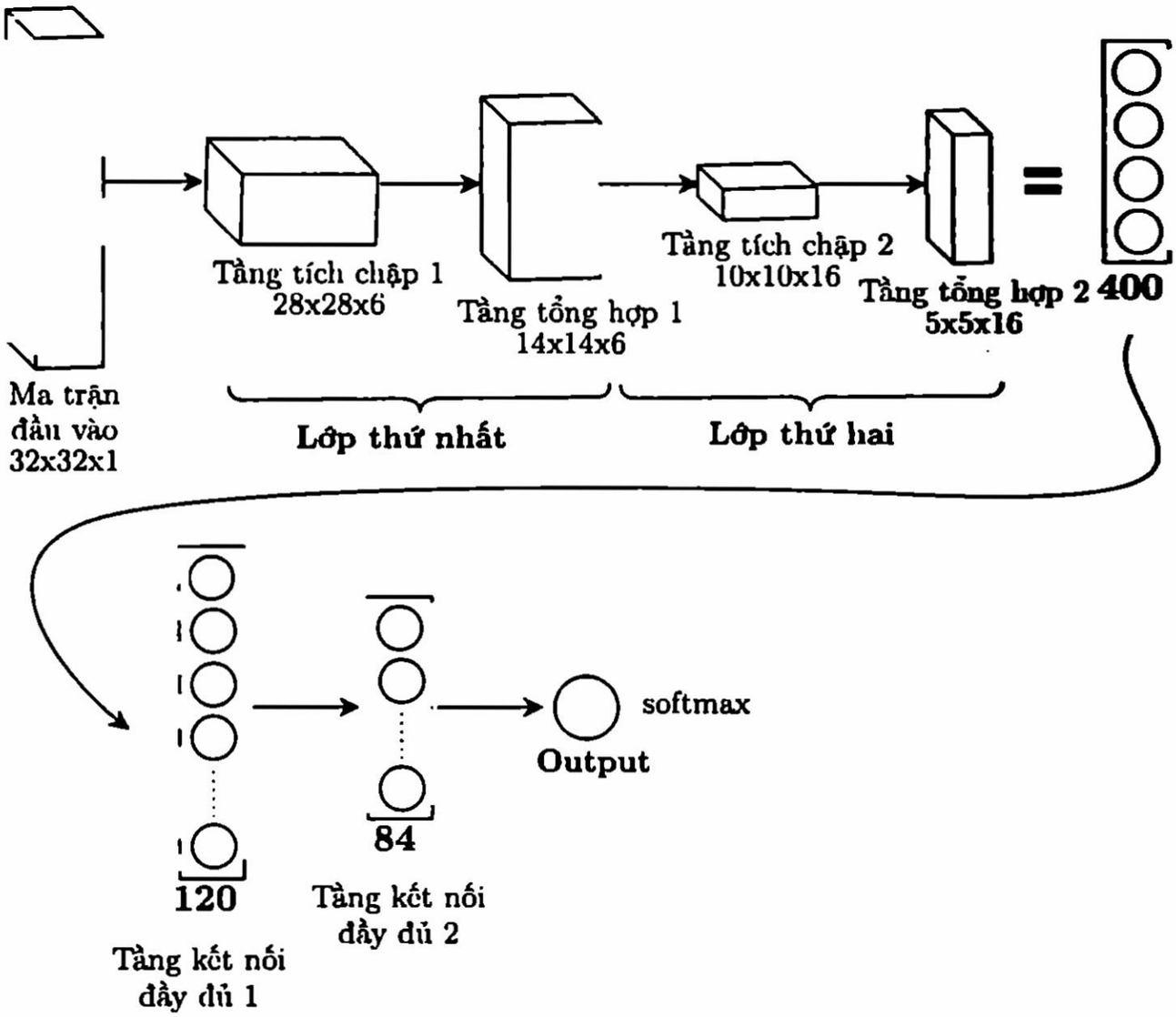
Hình 5.14 biểu diễn kiến trúc một mạng CNN để giải quyết bài toán phân loại. Trong đó bao gồm một ma trận đầu vào, một lớp tích chập và một lớp tổng hợp. Giả sử 40 là kết quả tính toán có được sau khi qua lớp tổng hợp (một vector 40 chiều), và vector 40 chiều này sẽ được đưa vào lớp kết nối đầy đủ, sử dụng hàm logistic hoặc softmax tùy theo mục đích, để thu được kết quả sau cùng.



Hình 5.14: Kiến trúc mạng CNN

Như phần trên đã đề cập, mạng CNN có thể kết nối nhiều tầng tích chập và tầng tổng hợp liên tiếp nhau để học các thuộc tính ẩn có tính trừu tượng cao. Tuy nhiên, kiến trúc mạng CNN phải luôn luôn kết thúc với một tầng kết nối đầy đủ nhằm thực hiện việc học và cập nhật trọng số thông qua cơ chế lan truyền ngược. Trong thực tế, hoàn toàn có thể dùng nhiều hơn một tầng kết nối đầy đủ nhằm tăng khả năng học, nhưng không nên dùng quá nhiều vì sẽ mất đi ý nghĩa của mạng CNN là giảm bớt số lượng của bộ trọng số cần phải học.

Mạng LeNet5 (Lecun Y. and P., 1998) là một trong những mạng CNN lâu đời và nổi tiếng nhất, được Yann LcCun phát triển vào những năm 1998. Cấu trúc của mạng LeNet5 gồm 2 lớp Convolution và Maxpooling, 2 lớp Fully Connected và output là hàm Softmax. Hình 5.15 thể hiện ví dụ một mạng LeNet5.



Hình 5.15: Kiến trúc mạng LeNet-5

Một số thông tin về kiến trúc mạng LeNet5 như sau:

- Đầu vào là một ma trận có kích thước $32 \times 32 \times 1$, nghĩa là một tấm ảnh có chiều dài 32 pixel, chiều rộng ảnh là 32 pixel và số lượng kênh ảnh là 1 (ảnh đen trắng).
- Lớp thứ nhất bao gồm:
  - Tầng tích chập thứ 1: là một ma trận $5 \times 5 \times 3$, tốc độ stride là 1, đi qua 6 cửa sổ trượt có kích thước $5 \times 5 \times 1$, và output của nó là 1 ma trận có kích thước $28 \times 28 \times 6$.
  - Tầng tổng hợp thứ 1: Kích thước cửa sổ trượt là 1 ma trận 2×2, tốc độ stride là 2, số lượng cửa sổ trượt là 16, và output của nó là 1 ma trận có kích thước $14 \times 14 \times 6$.

- Lớp thứ hai bao gồm:

  - Tầng tích chập thứ 2: là một ma trận $5 \times 5 \times 6$, tốc độ stride là 1, đi qua 16 cửa sổ trượt và output của nó là 1 ma trân có kích thước $10 \times 10 \times 16$.

  - Tầng tổng hợp thứ 2: Kích thước cửa sổ trượt là 1 ma trận 2×2, tốc độ stride là 2 và output của nó là 1 ma trận có kích thước $5 \times 5 \times 16$.

- Số nút Output của lớp thứ 2 sẽ là $5 \times 5 \times 16 = 400$.

- Số nút Output của tầng kết nối đầy đủ thứ 1 sẽ là 120.

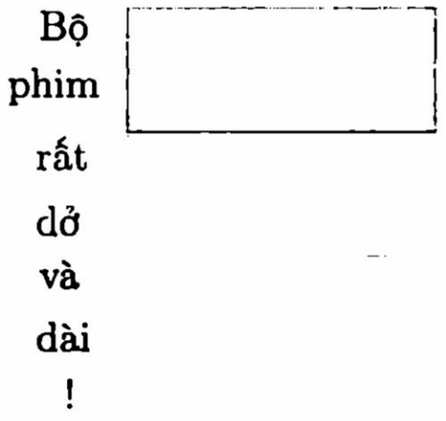
- Số nút Output của tầng kết nối đầy đủ thứ 2 sẽ là 84.

5.5 Sử dụng mạng CNN cho các bài toán NLP

Như đã trình bày trong Chương 4, *Xử lý ngôn ngữ tự nhiên* (Natural Language Processing - NLP) là một lĩnh vực nghiên cứu rất phong phú và sâu sắc, với rất nhiều kỹ thuật để trích xuất thông tin từ các ngôn ngữ. *NLP* có rất nhiều ứng dụng phổ biến bao gồm *phân loại tài liệu* (Text Classification), *nhận dạng giọng nói* (Speech Recognition) và *dịch vụ dịch thuật* (Translation Services). Với các bài toán này, ta hoàn toàn có thể sử dụng phương pháp cổ điển là Bag-of-Words, TF-IDF hay mô hình n-gram để giải quyết. Tuy nhiên, với việc xuất hiện của các phương pháp biểu diễn Word Embedding, chúng ta có thể sử dụng các mô hình học sâu như CNN để đưa ra các hướng tiếp cận hiện đại hơn.

5.5.1 Sử dụng CNN vào bài toán NLP với phương pháp Word Embedding

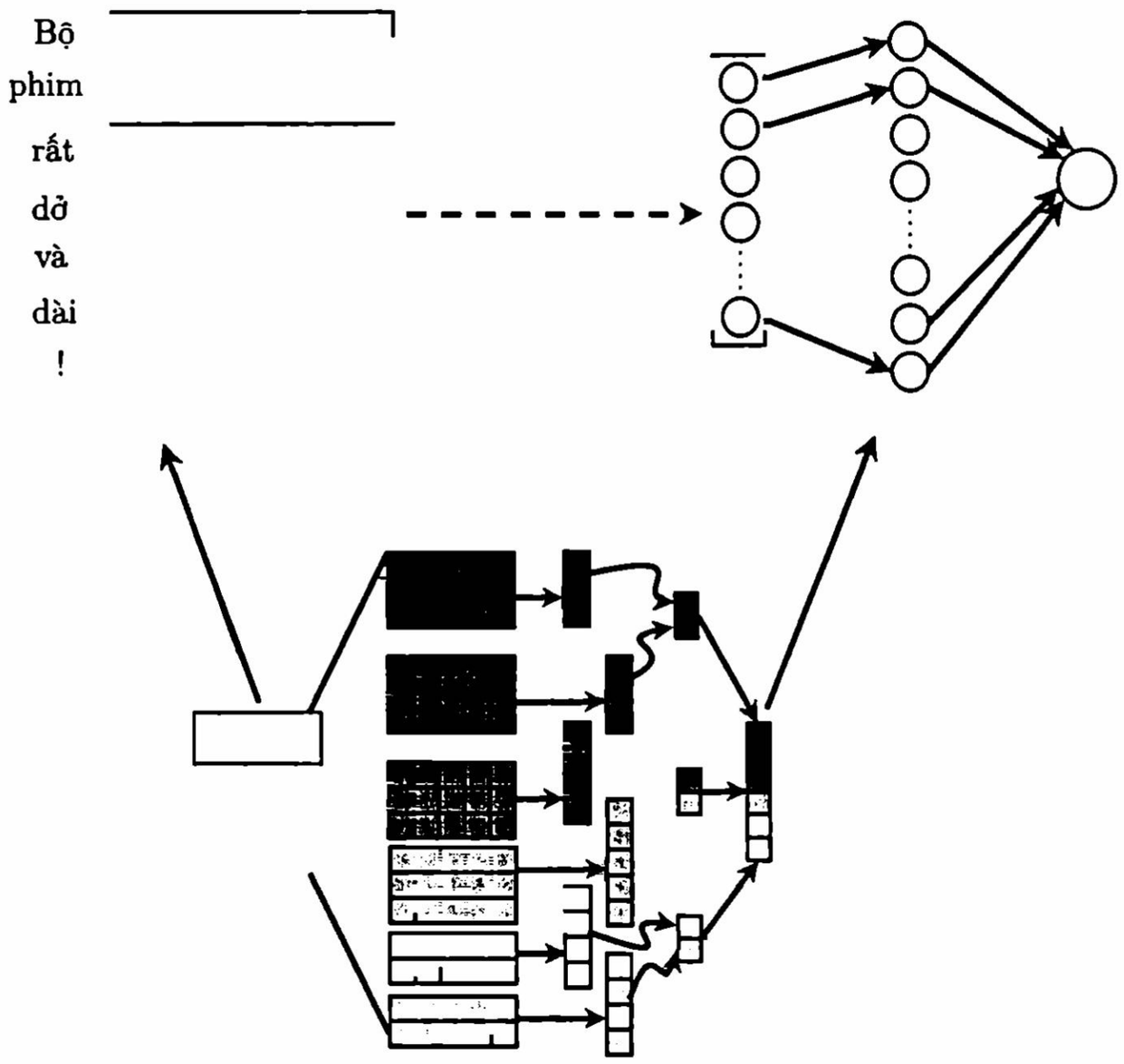
Với phương pháp Word Embedding, mỗi từ đều có thể biểu diễn thành một vector, từ đó một văn bản có thể giải thích thành một ma trận như Hình 5.16.



Hình 5.16: Biểu diễn mỗi từ trong câu thành một vector bằng Word Embedding

Từ Hình 5.16 trên, ta để ý rằng hình chữ nhật màu đỏ sẽ biểu diễn một bi-gram trên ma trận này. Tương tự, hình chữ nhật màu vàng biểu diễn một tri-gram. Như vậy một bộ lọc ma trận (filter) $2 \times k$ là phù hợp để biểu diễn và rút trích các latent feature của các bi-gram, tương tự một bộ lọc ma trận $3 \times k$ cho tri-gram.

Với ma trận từ được biểu diễn như Hình 5.16 ở trên, ta hoàn toàn có thể sử dụng như là input cho một mạng nơ-ron như bình thường, tuy nhiên với cách tiếp cận mới bằng các kỹ thuật Word Embedding, ta có thể sử dụng thêm tầng CNN để rút trích các latent features, được thể hiện như Hình 5.17 ở dưới, và cách biểu diễn kinh điển của một tầng CNN sẽ được trình bày trong case study tiếp theo.



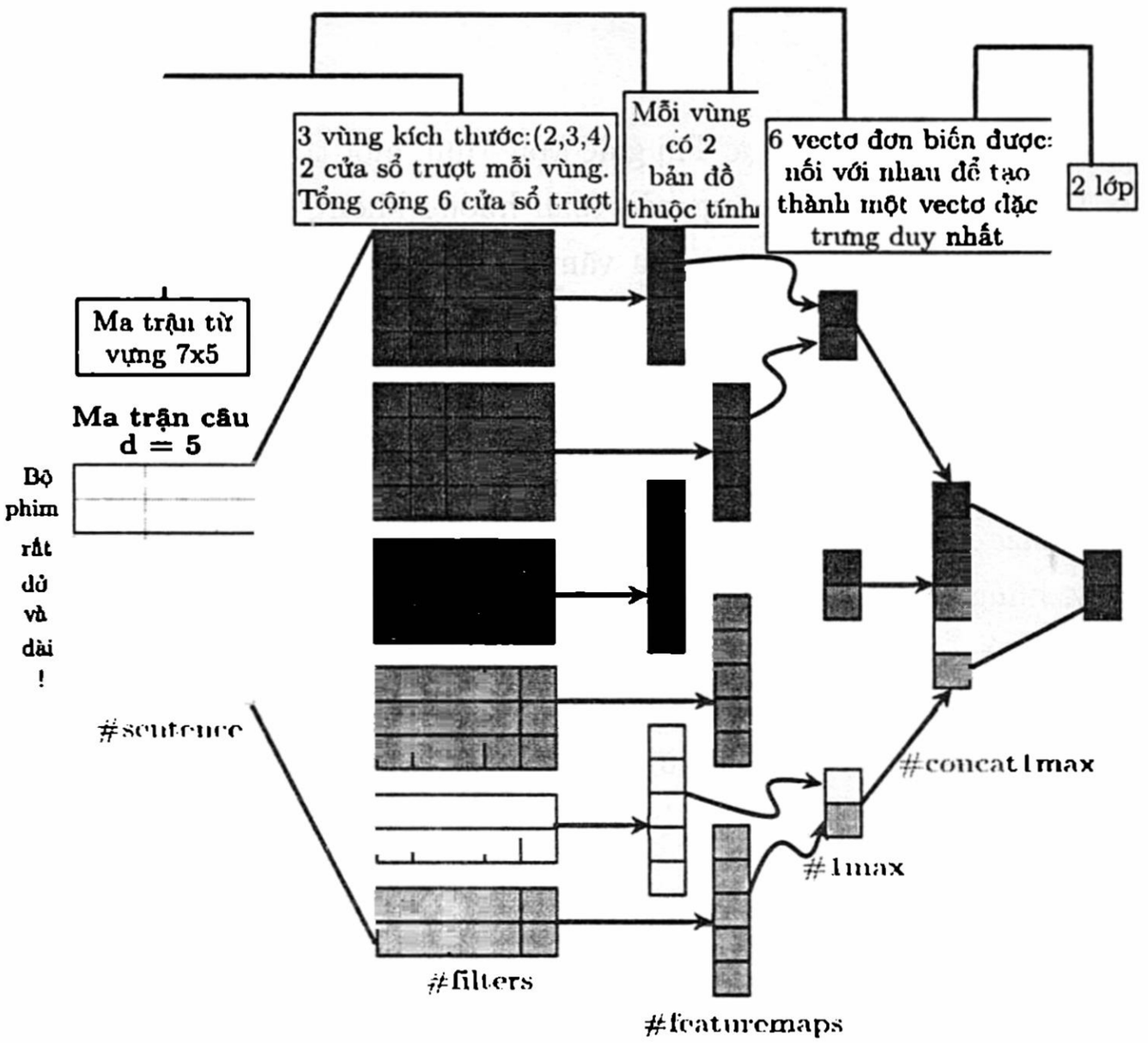
Hình 5.17: Sử dụng thêm tầng CNN trong cách tiếp cận mới

5.5.2 Case study: sử dụng CNN cho bài toán phân tích cảm xúc

Phân tích cảm xúc (sentiment analysis) là quá trình phân tích, đánh giá quan điểm, cảm xúc của một người về một đối tượng nào đó qua lời nói hoặc văn bản (quan điểm có thể mang tính tiêu cực, tích cực hay bình thường,...). Sử dụng các kỹ thuật NLP, *sentiment analysis* sẽ xem xét biểu thức của người dùng và lần lượt liên kết cảm xúc với những gì người dùng đã cung cấp. Tuy nhiên, cũng có trường hợp tùy thuộc vào nhiều yếu tố tác động mà một câu nói sẽ mang nhiều ý nghĩa cảm xúc khác nhau. *sentiment analysis* được áp dụng rộng rãi cho việc phân tích đánh

giá và phản hồi khảo sát, phương tiện truyền thông xã hội và trực tuyến và được sử dụng nhiều trong lĩnh vực dịch vụ chăm sóc khách hàng.

Trong phần này, chúng ta sẽ đi vào chi tiết một case study là sử dụng CNN cho bài toán *sentiment analysis*. Hình 5.18 là một sơ đồ biểu diễn kiến trúc CNN cho bài toán phân loại cảm xúc.



Hình 5.18: Sơ đồ biểu diễn kiến trúc CNN áp dụng cho bài toán phân loại cảm xúc

Để dễ hiểu, chúng ta sẽ chia sơ đồ Hình 5.18 ra thành 5 phần: #sentence, #filters, #featuremaps, #1-max và #concat-1-max.

#sentence

Thành phần này tạo ra ma trận biểu diễn câu. Ví dụ là câu *'Bộ phim rất dở và dài !'* tổng cộng có 7 từ trong câu (dấu chấm than vẫn được tính là một từ). Ở đây chúng ta chọn 5 là chiều của các vector từ. Chúng ta hãy biểu thị độ dài của câu là s và d là kích thước của vector từ, do đó chúng ta có một ma trận câu có hình dạng $s \times d$ là 7×5.

#filters

Khi sử dụng trong lĩnh vực Thị giác Máy tính, một trong những đặc tính của CNN là nó có khả năng bảo toàn hướng không gian 2D trong ảnh gốc. Tuy. nhiên, đối với tài liệu văn bản thì dữ liệu thường được xem là có cấu trúc một chiều để biểu thị trình tự các từ. Trong tình huống này, vì chúng ta đã biểu diễn các từ trong câu ví dụ bằng các vector từ 5 chiều, do đó chúng ta sẽ sửa một chiều của bộ lọc để khớp với vector từ và thay đổi kích thước vùng *h (region size)*, trong đó số hàng của *region size* sẽ tương ứng với số từ trong ma trận câu sẽ được lọc. Cụ thể ở đây,chúng ta sẽ chọn kích thước cho các bộ lọc được biểu diễn bởi các ma trận có số cột là 5, số hàng sẽ tùy biến (2, 3, 4) ứng với các thông tin về bigram, trigram và 4-gram của văn bản. Ứng với mỗi số hàng, chúng ta sẽ áp dụng 2 bộ lọc cho từng vùng. Như vậy, sau bước convolution đầu tiên ta sẽ có tổng cộng 6 bộ lọc.

#featuremaps

Đối với phần này, chúng ta sẽ sử dụng CNN để thực hiện các phép toán tích chập để tạo ra các bản đồ thuộc tính như đã trình bày trước đó.

#1-max

Ta đã biết được rằng chiều của c phụ thuộc cả s và h, nói cách khác, nó sẽ khác nhau giữa các câu có độ dài khác nhau và các bộ lọc có kích thước vùng khác nhau. Để giải quyết vấn đề này, chúng ta sẽ sử dụng hàm gộp 1-max và trích xuất số lớn nhất từ mỗi vector c.

#concat-1-max

Sau khi thực hiện phép gộp 1-max, chúng ta có được một vector có độ dài cố định gồm 6 phần tử (*số bộ lọc cho mỗi kích thước vùng (2)* x *số lượng kích thước vùng được xem xét (3)*). Vector có chiều dài cố định này sau đó có thể được đưa vào một lớp softmax *(fully-connected)* để thực hiện việc phân loại. Ở đây chúng ta giả định là phân loại nhị phân và do đó sẽ có hai trạng thái đầu ra. Sự khác biệt giữa kết quả của hàm softmax và giá trị mong đợi sẽ được lan truyền ngược để cập nhật các trọng số sau:

- Ma trận w (ma trận dùng để tạo ra ma trận o).
- Bias được thêm vào o để tạo ra ma trận c.
- Các vector từ.

Cần lưu ý rằng vector kết quả của softmax vẫn có thể sử dụng được cho các phương pháp học máy cổ điển khác như SVM (Support Vector Machine) hay Naive Bayes để thực hiện việc phân loại. Tuy nhiên, trong trường hợp này, nhiều mô hình hiện thực sử dụng một tầng fully connected (tức là qua một mạng Nơ-ron đơn giản). Điều này sẽ giúp chúng ta sẽ có một mạng *end-to-end*, đầu vào và đầu ra của hệ thống đều được kết nối qua một kiến trúc mạng nơ-ron. Nhờ vậy, sau khi có kết quả mất mát ở tầng cuối cùng, hệ thống có thể cập nhật toàn bộ các trọng số từ đầu bằng cơ chế backpropagation. Toàn bộ các bước đều có thể học theo cơ chế Backpropagation này, do đó trọng số của các filter tương ứng với n-gram, *trigram*, *bigram*,... đều được học và cập nhật liên tục như đã trình bày trên.

Trong thực tế, để lựa chọn kích thước và số lượng các bộ lọc cho các bài toán NLP áp dụng mạng CNN, chúng ta thường phải tham khảo từ các bài báo liên quan nổi tiếng trong ngành. Ở đây, chúng tôi giới thiệu một trong các bài báo tiêu biểu về việc lựa chọn các siêu tham số *(hyperparameters)*như trong ví dụ trên, bài báo *'Convolutional Neural*

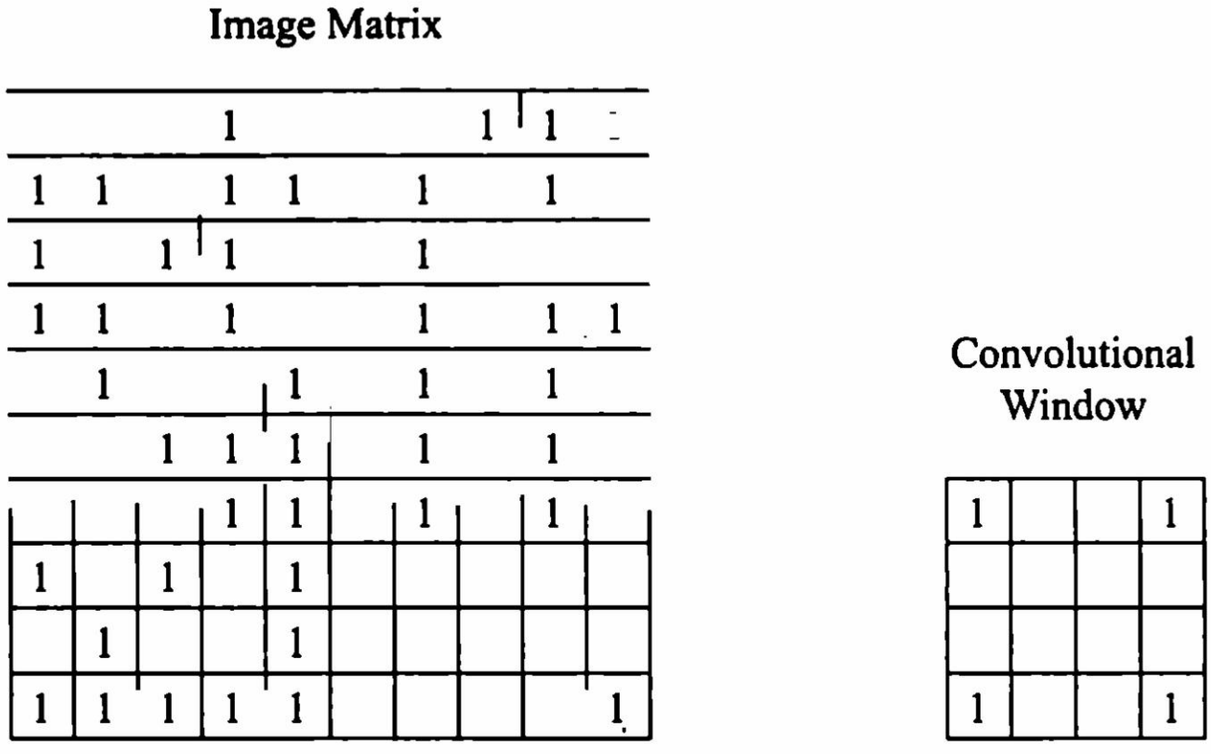
Networks for Sentence Classification' của tác giả Yoon Kim (2014) (Kim, 2014). Trong bài báo, tác giả đã mô tả một loạt các thí nghiệm với các mạng CNN được xây dựng trong mô hình word2vec. Mặc dù có một ít sự điều chỉnh các *hyperparameter*, nhưng tác giả đã thu được kết quả là một mạng CNN đơn giản với một lớp chập thực hiện rất tốt. Kết quả của tác giả đã khẳng định cho một luận điểm rõ ràng rằng việc huấn luyện trước *(pre-training)* các vector từ không được giám sát *(unsupervised)* là một phần quan trọng trong việc ứng dụng học sâu cho NLP.

Cụ thể, qua tất cả các thực nghiệm về việc lựa chọn *hyperparameter* trong bài báo, tác giả đã sử dụng hàm kích hoạt *rectified linear units*, cửa sổ bộ lọc (h) 3, 4, 5 đối với mỗi 100 feature map, tỉ lệ bỏ qua *(dropout rate)* là 0.5, ràng buộc L_2 cho các hàng của ma trận *softmax* s là 3 và mini-batch size = 50. Các giá trị này được chọn thông qua tìm kiếm dạng lưới *(grid search)* trên bộ huấn luyện SST-2 (Stanford Sentiment Treebank - đây là một phần mở rộng của thông số MR, Movie review với mỗi đánh giá tích cực/tiêu cực bằng một câu, nhưng có thêm các phân tách train/dev/test được cung cấp và các nhãn đánh giá (*very positive*, *positive*, *neutral*, *negative*, *very negative*)).

Đối với thông số chiều cho các vector từ, ở đây tác giả tin rằng việc khởi tạo vector từ với những từ thu được từ mô hình ngôn ngữ nơ-ron không giám sát *(unsupervised neural language model)* là một phương pháp phổ biến để cải thiện hiệu suất khi không có bộ dữ liệu được gán nhãn lớn. Tác giả sử dụng các vector Word2Vec có sẵn công khai được huấn luyện trên 100 tỷ từ trên Google News. Các vector này có chiều là 300 và được huấn luyện bằng cách sử dụng kiến trúc CBOW *(continuous bag-of-words)*. Các từ không có trong tập hợp *pre-trained words* sẽ được khởi tạo ngẫu nhiên. Trong bài báo này, tác giả cũng nhấn mạnh rằng trong suốt quá trình huấn luyện, cần luôn tiếp tục kiểm tra hiệu suất trên bộ dữ liệu huấn luyện và chọn trọng số chính xác cao nhất *(highest accuracy weights)* cho lần đánh giá cuối cùng.

5.6 Bài tập

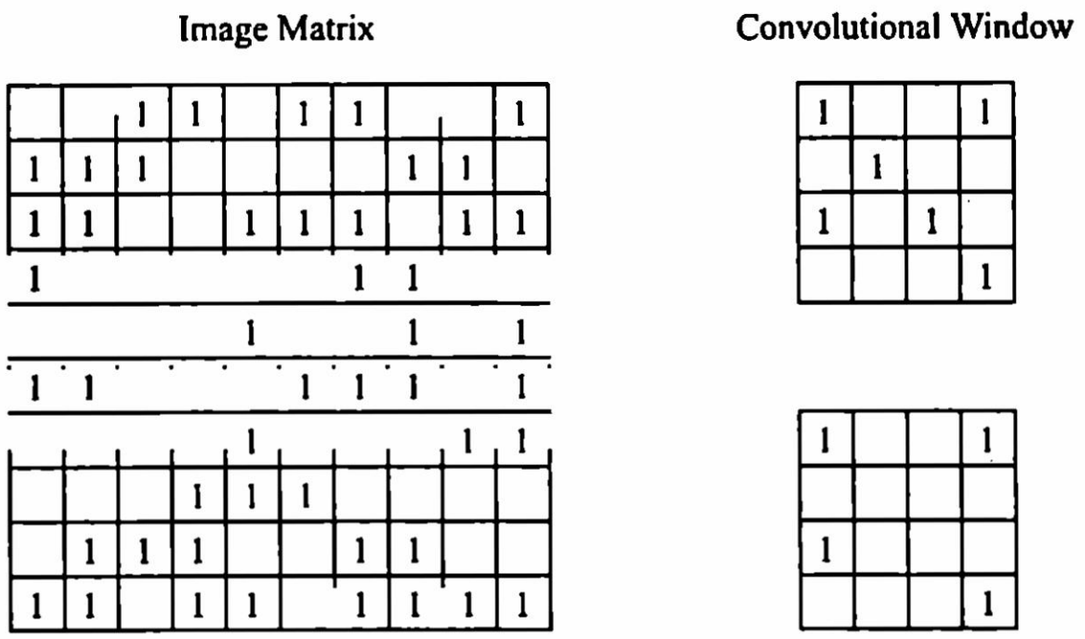
Bài tập 5.6.1. Cho ma trận hình ảnh kích thước 10×10 và một cửa sổ tích chập kích thước 4×4 như Hình 5.19. Hãy cho biết ma trận kết quả cụ thể và số chiều của ma trận này sau khi thực hiện phép tích chập và phép gộp max pooling.



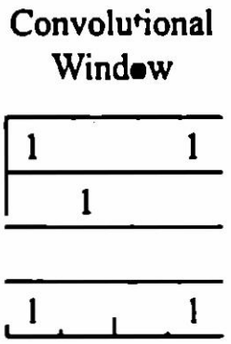
Hình 5.19: Ma trận hình ảnh và cửa sổ tích chập trong bài tập 5.6.1

Bài tập 5.6.2. Cho một mạng nơ ron có 3 tầng, với tầng thứ nhất là một ma trận kích thước 10×10 và có 2 cửa sổ trượt ở tầng 1 như Hình 5.20 và ở tầng thứ hai có 1 cửa sổ trượt như Hình 5.21. Hãy cho biết ở tầng thứ 3 sẽ là vector có bao nhiêu chiều?

Bài tập 5.6.3. Với cách làm cho Bài tập 5.6.2 thì ta vẫn cần ma trận ban đầu có số chiều cố định. Nếu chiều dài không cố định thì cần xử lý như thế nào?



Hình 5.20: Ma trận hình ảnh và 2 cửa sổ tích chập ở tầng đầu tiên trong Bài tập 5.6.2



Hình 5.21: Cửa sổ tích chập cho tầng thứ 2 trong Bà. tập 5.6.2

Bài tập 5.6.4. Cho câu sau đây: *Bộ phim rất dở và dài !*

One-hot vector của câu này sẽ là

	Bộ	phim	rất	dở	và	dài	!
Bộ	1	0	0	0	0	0	0
phim	0	1	0	0	0	0	0
rất	0	0	1	0	0	0	0
dở	0	0	0	1	0	0	0
và	0	0	0	0	1	0	0
dài	0	0	0	0	0	1	0
!	0	0	0	0	0	0	1

Ma trận Embedding của câu này sẽ là

$$\begin{bmatrix} 8 & 2 & 1 & 9 \\ 2 & 4 & 7 & 0 \\ 1 & 3 & 5 & 8 \\ 0 & 1 & 9 & 4 \\ 7 & 8 & 1 & 2 \\ 1 & 5 & 8 & 9 \\ 5 & 1 & 5 & 2 \end{bmatrix}$$

Hãy cho biết ma trận biểu diễn câu này như thế nào ?

Bài tập 5.6.5. Cho ma trận ban đầu là

$$\begin{bmatrix} 6 & 5 & 9 & 1 \\ 3 & 4 & 7 & 1 \\ 1 & 7 & 5 & 1 \\ 0 & 7 & 9 & 0 \\ 4 & 2 & 2 & 4 \\ 7 & 5 & 9 & 8 \\ 1 & 1 & 1 & 0 \end{bmatrix}$$

Ma trận 2-gram là $\begin{bmatrix} 1 & 5 & 9 & 3 \\ 3 & 8 & 0 & 2 \end{bmatrix}$

Ma trận 3-gram là $\begin{bmatrix} 0 & 7 & 9 & 4 \\ 2 & 9 & 7 & 5 \\ 3 & 3 & 0 & 7 \end{bmatrix}$

Bằng cách sử dụng Max pooling, hãy cho biết bản đồ thuộc tính sau cùng được tạo ra.